Nama : Muhammad Rizal Thoriq
Kelas :1H
No. Presensi : 24

Percobaan 1



```java
import java.util.Scanner;

public class Star24 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            System.out.print("*");
        }
        System.out.println();
        sc.close();
    }
}
```

1. Jika inisialisasi i = 1 diubah menjadi i = 0 tanpa mengubah kondisi, berapa karakter * yang tercetak ketika n = 5? Jelaskan.
2. Jika kondisi i <= n diganti menjadi i < n, berapa kali perulangan dijalankan saat n = 5? Mengapa hasilnya berbeda?
3. Prediksi output potongan kode berikut ketika n = 9, kemudian buktikan dengan mengganti perulangan FOR pada program Anda dan menjalankannya.
   for (int i = 2; i <= n; i += 2) { System.out.print(i + " "); }
   Modifikasi kode tersebut agar mencetak bilangan ganjil positif yang tidak melebihi n.
4. Jelaskan mengapa perubahan increment i++ menjadi i-- (dengan kondisi tetap i <= n) membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar untuk n positif. Jangan menjalankan versi tersebut; jika terlanjur berjalan, hentikan program dengan menekan Ctrl + C pada terminal.

Jawaban:

1. 6 karakter, karena perulangan dimulai dari i = 0 sampai i <= 5 yang dimana nilai i akan mengalami perulangan sebanyak 6 kali.
2. Perulangan akan terjadi sebanyak 4, karena jika diganti operator < saat i mencapai nilai n (5 < 5) merupakan kondisi false
3. Outputnya akan menjadi “2 4 6 8” karena i dimulai dari 2 dan kemudian ditambah 2 setelah mengalami perulangan. Untuk mencetak angka ganjil kodenya menjadi
   for (int i = 1; i <= n; i += 2) { System.out.print(i + " "); }
4. Hal ini disebabkan karena jika diisi *i*— saat i dimulai dari 1 dengan kondisi *i* <= *n*. Nilai i akan akan semakin mengecil pada setiap iterasi sehingga kondisi i <= n selalu bernilai true.

Nama : Muhammad Rizal Thoriq
Kelas :1H
No. Presensi : 24

Percobaan 2



1. Tentukan fungsi variabel i dan j pada program tersebut
2. Berapa kali outer loop, inner loop, dan perintah print("*") dijalankan untuk baris = 4 dan kolom = 7?
3. Apa yang terjadi jika batas perulangan luar dan dalam ditukar (outer menggunakan kolom, inner menggunakan baris)? Gunakan baris = 3 dan kolom = 6 untuk menjelaskan.
4. Jelaskan akibat penghapusan System.out.println(); setelah inner loop
5. Modifikasi program agar karakter yang dicetak adalah #, lalu commit dan push perubahan ke GitHub.

Jawaban

1. Variabel i sebagai outer loop yang berfungsi mengatur dan menghitung jumlah baris pada program.
   Segangkan variabel j sebagai inner loop yang berfungsi sebagai pengatur banyak karakter yang dicetak kesamping
2. Pada outer loop terjadi looping sebanyak 4 kali. Sedangkan pada inner loop sebanyak 7 kali, jika total looping pada inner loop terjadi sebanyak 28 kali.
3. Bentuk pola persegi panjang bintang akan terbalik atau berputar. Jumlah baris output yang dihasilkan akan mengikuti nilai kolom (yaitu 6 baris), dan jumlah bintang ke samping di tiap barisnya akan mengikuti nilai baris (yaitu 3 bintang per baris). Jika posisi ditukar, *outer loop* berjalan sebanyak 6 kali (membentuk 6 baris), dan *inner loop* mencetak 3 bintang per barisnya. Output akhirnya menjadi:
   ***
   ***
   ***
   ***
   ***
   ***
4. Jika System.out.println(); setelah inner loop, yang terjadi adalah semua karakter akan tercetak kesamping menjadi baris panjang tanpa terputus.

Nama : Muhammad Rizal Thoriq
Kelas :1H
No. Presensi : 24

Percobaan 3

```java
import java.util.Scanner;

public class Triangle24 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("masukkan nilai n = ");
        int n = sc.nextInt();

        int i = 0;
        while (i <= n) {
            int j = 0;
            while (j <= i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i++;
        }
        i = n;
        while (i >= 0) {
            int j = 0;
            while (j <= i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i--;
        }
        sc.close();
    }
}
```

1. Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan untuk menghasilkan pola segitiga bertingkat.
2. Jelaskan peran variabel i dan j. Mengapa nilai j harus diinisialisasi kembali ke 0 pada setiap iterasi outer loop? Apa yang terjadi jika j tidak di-reset?
3. Jelaskan perubahan inisialisasi, kondisi, dan perubahan nilai (update) pada outer loop yang diperlukan untuk membentuk segitiga terbalik

Jawaban

1. Pada kode awal, semua bintang dicetak dalam satu baris menyambung tanpa ada baris baru, serta terdapat baris kosong di awal karena iterasi dimulai dari i = 0 dengan j < 0 yang tidak mencetak bintang sama sekali.
   Yang pertama menambahkan System.out.println(); setelah *inner loop* selesai (sebelum i++) agar kursor berpindah ke baris baru pada setiap iterasi baris.
   Yang kedua mengganti inisialisasi awal int i = 1; atau mengubah kondisi *inner loop* menjadi j <= i agar baris pertama langsung mencetak 1 bintang dan tidak ada baris kosong.
2. Variabel i sebagai pengontrol *outer loop* yang menentukan nomor baris dan membatasi jumlah maksimal bintang yang harus dicetak pada baris tersebut. Sedangkan variabel j sebagai pengontrol *inner loop* yang menghitung jumlah bintang yang sedang dicetak pada baris berjalan.
   Agar setiap kali berpindah ke baris baru, penghitungan jumlah bintang harus dimulai dari awal lagi. Yang terjadi jika j tidak di reset Nilai j akan tetap menyimpan nilai terakhir dari baris sebelumnya (sehingga nilai j sudah sama atau lebih besar dari i). Akibatnya, *inner loop* pada baris-baris berikutnya tidak akan pernah berjalan lagi dan tidak ada bintang baru yang tercetak.
3. Inisialisasi (i = n): Variabel i dimulai dari nilai n (jumlah bintang terbanyak di baris paling atas), bukan lagi dari 0 atau 1.
   Kondisi (i >= 1 atau i > 0): Kondisi diubah agar perulangan tetap berjalan selama i masih lebih besar atau sama dengan 1.
   Perubahan Nilai / Update (i--): Operator diubah dari penambahan (i++) menjadi pengurangan (i--) agar jumlah bintang di baris berikutnya selalu berkurang 1 hingga mencapai 1 bintang.

Nama : Muhammad Rizal Thoriq
Kelas :1H
No. Presensi : 24

Percobaan 4

1. Mengapa totalNilai perlu di-reset di dalam outer loop, tetapi rataTertinggi tidak boleh di-reset pada setiap kelompok?
2. Untuk 3 kelompok dan 2 penilai (tanpa input yang salah), berapa kali badan perulangan FOR dijalankan secara keseluruhan?
3. Apa yang terjadi jika totalNilai dideklarasikan sebagai int dan rata-rata dihitung menggunakan pembagian dua bilangan bulat? Berikan contoh nilai.
4. Mengapa rataTertinggi diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah apabila nilai awalnya 0.
5. Program ini memiliki tiga tingkat perulangan. Sebutkan masing-masing jenis perulangan beserta perannya.

Jawaban

1. totalNilai harus di-reset ke 0 di setiap kelompok agar akumulasi nilai antar kelompok tidak tercampur. Sebaliknya, rataTertinggi tidak boleh di-reset agar rekor nilai rata-rata tertinggi dari seluruh kelompok tetap tersimpan.
2. Totalnya sebanyak 6 kali
3. Angka di belakang koma (desimal) dari hasil pembagian rata-rata akan terpotong, sehingga akurasi nilai rata-rata hilang.
4. Agar pada kelompok pertama, nilai rata-ratanya pasti lebih besar dari -1 sehingga kelompok pertama otomatis tercatat sebagai kelompok terbaik awal. Jika diisi 0, kelompok dengan rata-rata 0 tidak akan terekam dengan benar.
5. WHILE luar (mengatur perulangan kelompok), FOR tengah (mengatur perulangan penilai), dan DO-WHILE dalam (sebagai validasi input nilai agar berada di rentang 0–100).